01 运营体系正向设计基础概念 · 参考答案

01 - 运营体系正向设计基础概念 · 参考答案

> **对应课件**：`01-运营体系正向设计基础概念`（10 组 × 5 问 = 50 题）
> **编制依据**：**库内标准正本**——ISO 9000:2026、ISO/IEC/IEEE 15288:2023、ISO/IEC/IEEE 42010:2022、ISO/IEC/IEEE 42020:2019、ISO/IEC/IEEE 42030:2020；**库外标准**（本库无正本，正文标〔库外〕）——ISO 9001:2015、ISO/IEC/IEEE 29148:2018、ISO 10303-1:2021、ISO 37000:2021、ISO/IEC 38500:2024、ISO 1087:2019、IEEE 1028-2008、IEEE 610.12-1990。
> **怎么读**：每题三段——【答案】先用大白话说结论，标准原文放进括注；【解析】讲清为什么；【示例】给华翔的例子。凡是本课程自己编的讲法（标准里没有这个词），句中都会直接说明"这是本课的讲法"，不与标准条文混为一谈。

> ⚠️ 本目录同名 `.pdf` / `.key` / `.pptx` 系**本轮修订前**生成的构建产物，**内容尚未同步**——以 `.md` 为准，重出后再引用。

> **几处译名先交代**：本件说的"架构"一律按 ISO/IEC/IEEE 42010:2022 §3.2 的口径；旧译名与今译名的对应是——视点→**架构视角**、concern→**关注点**、entity of interest→**所关注实体**、相关方→**利益相关方**、生命周期→**生存周期**、治理第三成分→**担责**。另：ISO 9000 一律按 **2026 版**条款号（需求 §3.5.1、质量 §3.5.2、设计和开发 §3.3.11、验证 §3.11.12、确认 §3.11.14、目标 §3.7.11；评审 §3.11.2 两版同号）。

贯穿案例：华翔研发运营体系（50 个示例的共同底稿）

> 本讲 50 道示例共用**一家企业、一套体系、一批带编码的工件、一组固定的人**。示例按题目顺序展开，开头用〔案例·××〕标出它落在体系的哪个位置，结尾落到具体工件上。把十组连起来读，就是"看一家企业把自己的运营体系正向设计出来、建起来、转起来"的完整一条线索——术语不再是散的，而是同一条流水线上各个位置的名字。

一、企业与业务

华翔航空电子有限公司（简称华翔）做机载显示、通信导航与任务计算机，客户是主机厂和机队运营方，受适航规章约束。2025 年起战略从"零部件配套"转向"系统级供应商"：型号变多、变更更频繁、适航证据要可追溯、外场排故要更快。压力全落在研发运营上，于是有了两个开发对象——**华翔研发运营体系**（制度＋流程＋组织＋指标＋IT 需求），以及承载它的**统一研发平台 HDP**。

二、这套体系由什么组成（四根柱子）

组成	在华翔是什么	谁负责
流程体系	需求→设计→试制→验证→批产移交→外场保障，逐层细化到 L3 作业步骤	流程工程师 老张
人员与组织	型号团队、专业室、评审专家库、工艺与试验队伍	组织设计 DS-04
工具与技术	统一研发平台 HDP、CATIA/CAE、试验台、数据与模型	IT 架构师 小陈
指标体系	任务完好率、交付准时率、变更响应、外场排故时长	运营管理部 周经理

三、一条生命线：三态与关键站

状态	走到哪一步	主要工件
设计态（应然·图纸）	定目的→做需求→定架构→做设计，直到八方案齐备	SP-01、PU-01、RQ-01/02/03、CN-01、GO-01、VS-01、CP-01、AD-01、ADR-01、GP-01、IC-01、VP-01、DS-01~06、TE-01…MT-01
生产态（应然→实然）	制度颁布、流程上线、组织到位、HDP 建成	颁布件、上线记录、DS-04 调整令
运行态（实然·运转）	流程在跑、看板在刷、偏差进改进	KPI-01、CR-01、CI-01、VR-01、VA-01

四、五方与人物

方	在案例里是谁
所有方	董事会（董事长 周振华）＋研发运营体系委员会（＝架构委员会，即治理机构下的"设计权威机构"）
需求方	总经理 李承志、各业务域、主机客户代表 王工、适航委任代表 何工
开发方	流程与数字化团队：流程工程师 老张、业务分析 小赵、IT 架构 小陈
使用方	设计员 小林、工艺员 老吴、试验员 小许、外场排故 老黄
运维方	运营管理部 周经理、平台运维 小唐

五、工件编码（示例里出现时不再解释）

> **编码怎么读**：两位字母＝英文词头（有的是取词中两个关键字母，如 **RQ** 取 Re**q**uirement、**RV** 取 Re**v**iew），短横后两位是序号；同一族里序号只表示第几件，不表示先后关系。

编码	编码由来（英文词头）	中文名称	属于哪一态
SP-01	**S**trategic **P**lan 战略计划	战略解码与年度经营目标	设计态·最上游
PU-01	**P**urpose **U**nderstanding 对目的与任务的理解确认	系统目的或任务理解确认单	设计态·最上游
RQ-01	**Req**uirement 需求（利益相关方层）	利益相关方需求规格	设计态
RQ-02	同上（系统层）	系统需求规格	设计态
RQ-03	同上（已规定成文）	规定需求清单（含 AS-xxx 适航条款摘录）	设计态
CN-01	**Concen** 关注点（42010 术语）	关注点台账	设计态·架构
GO-01	**G**oal 目标	架构目标与关键成功准则	设计态·架构
VS-01	**V**iewpoint **S**pecification 视角规格（42010 术语）	视角规格集（项目 ADF）	设计态·架构
CP-01	**C**oncepts & **P**roperties 基本概念与基本属性（42010 §3.2 定义句里的两样东西）	基本概念和基本属性清单	设计态·架构
AD-01	**A**rchitecture **D**escription 架构描述（42010 标准术语）	架构描述	设计态·架构
ADR-01	**A**rchitecture **D**ecision **R**ecord 架构决策记录	架构决策记录	设计态·架构
GP-01	**G**overning **P**rinciples 管控原则（42010 定义句后半）	管控原则	设计态·架构
IC-01	**I**nterface **C**ontract 接口契约	系统接口契约	设计态·架构
VP-01	**V**erification **P**lan 验证计划	系统级验证方案	设计态·架构
DS-01…06	**D**esign **S**pecification 设计交付物集（01–06 只表序号）	业务蓝图 / 流程文件集 / 制度文本 / 组织设计 / 指标体系 / IT 需求规格	设计态·设计输出
TE / PR / MF / TS / DL / DP / OP / MT-01	**T**echnical / **P**urchase / **M**anufacturing / **T**est / **D**elivery / **D**eployment / **O**peration / **M**aintenance	技术 / 采购 / 制造（工艺） / 测试 / 交付 / 部署 / 操作 / 运维 八个方案	设计态·设计终点
RV-01	**Rev**iew 评审	评审记录	设计态出口
VR-01	**V**erification **R**ecord 验证记录	验证证据包	设计态出口

编码	编码由来（英文词头）	中文名称	属于哪一态
VA-01	**V**alidation 确认	确认报告	运行态入口
BL-01	**B**asel**i**ne 基线	需求基线	运行态
KPI-01	**K**ey **P**erformance **I**ndicator 关键绩效指标	研发运营看板	运行态
CR-01	**C**hange **R**equest 变更请求	变更单	运行态
CI-01	**C**ontinual **I**mprovement 持续改进	持续改进项	运行态

六、示例路线图：哪一组讲哪一段

组	讲的是	在生命线上的位置	代表工件
1 运营体系是什么	体系全景、三态、手段、五方、方法论	全局俯瞰	SP-01、PU-01、DS-01~06
2 产品与产品数据	线上流的是什么、职责怎么定、什么算增值	全程主干	RQ、AD、DS、VR、BL-01
3 需求与质量	三形态、规定化、质量尺子、判据、质量与进度	设计态·入口	RQ-01/02/03
4 设计与开发	要求变详细、起点终点、八方案、工艺	设计态·主体	CP、DS、TE…MT
5 工程	工程是什么、工程化、能力与角色	人的支撑面	规程、台账、发布规程
6 目的目标与需求	目的从哪来、目标怎么量、链条怎么追	设计态·最上游（时间上最早）	SP-01、PU-01、RQ
7 评审确认验证	三把尺子各管一段	设计态出口 + 运行态入口	RV-01、VR-01、VA-01
8 架构与视图	架构是什么、基本、视图、输入输出、清单	设计态·承重墙	CP-01、AD-01、ADR-01、GP-01
9 治理与架构治理	谁定方向、谁担责、委托与分工、架构治理	悬在全程之上	委员会决议、审计记录
10 学习与概念	概念、概念体系、怎么算掌握	认知底座（贯穿）	术语表、培训记录

> **阅读提示**：时间顺序上，"目的→需求→架构→设计→验证→运行"是正序；题目顺序是"先俯瞰、再拆讲"。两种顺序在示例里用同一个案例对齐，随时可以回到本页对位。

一、运营体系是什么

1. 什么是运营，什么是体系，什么是运营体系，运营体系的目的是什么

答案： 运营，就是把进来的东西变成能交付的成果的一连串活动——市场、研发、生产、售后、管理都算（运营管理学通说，Heizer & Render；标准里最接近的是"过程"：利用或转换输入、交付结果的一组活动，ISO 9000:2026 §3.3.1）。体系，就是一堆相互关联、相互作用的要素凑成的整体（§3.4.1）。运营体系 = "干活的活动" + "干活要用的家伙"，两样合起来才叫体系（本课的讲法，标准里没有"运营体系"这个词）。它的目的分两层：眼前是创造价值、提高生产率；长远是当战略和日常运转之间那座桥。拿什么量它？看五件事——质量、可靠、速度、柔性、成本（Slack 的"沙锥"五目标）：客户盯前三件，企业盯后两件，按竞争策略取舍。

解析： 运营和体系要拆开看：运营回答"做什么"，体系回答"由什么凑成"，少一个都不成立。五目标为什么能当尺子？因为它量的正是运营有没有兑现目的：做对（质量）、按时（可靠）、做快（速度）、应变（柔性）、做省（成本）。还要记住一点：五目标没有一项能全拿，企业是用策略在五者之间取舍，不是五项都要满分。

示例： 〔案例·全景〕华翔航空电子做机载显示、通信导航、任务计算机，客户是主机厂和机队。把图纸、试验数据变成"能装机、能飞"的设备，这是运营；研发、试制、批产、外场保障这些流程，加上统一研发平台 HDP、工艺装备、数据与技能，合起来才叫运营体系。体系好不好就看五件事：装机能不能用、到手快不快、交付准不准、改型灵不灵、单台成本高不高。

2. 业务需求、合规要素、制度流程、IT系统及运行，哪些属于运营体系

答案： 全算。但它们不是"属于/不属于"的逐项选择，而是同一件产品在三个状态上的不同样子（本课的"三态"讲法）。设计态就是图纸：业务需求说要创造什么、合规要素划红线、制度流程规定怎么干，此时都还是文档。生产态是照图施工：制度颁布、流程上线、组织到位、IT 系统建成。运行态是真转起来：流程在跑、制度在守、绩效在量。题干里的"IT 系统及运行"横跨后两个状态——建成算生产，跑起来算运行。

解析： 为什么要分三态？因为"建成了"不等于"用上了"，"用上了"也不等于"管用"。三态缺一段，都不算建成运营体系——只写制度、只建系统，都还停在前两态。标准里这套对应关系是现成的：设计态≈业务或任务分析（15288:2023 §6.4.1）到设计定义（§6.4.5）；生产态≈实现、集成、验证、转换、确认（§6.4.7–§6.4.11）；运行态≈运行、维护（§6.4.12–§6.4.13）。

示例： 〔案例·三态〕还是那五类东西，各在各的状态：需求规格、合规清单、制度与流程文件（RQ-01/03、DS-02/03）还在设计态；制度颁布、流程上线、组织到位、HDP 建成属于生产态；流程在跑、看板在刷新、绩效在考核才是运行态。所以"系统建成了"只到生产态，"业务问题解决了"才算运行态。

3. 支持管理业务的信息系统属于运营体系，研发所用的CAX工具算不算

答案： 算。判断不看你用的是什么工具，只看它是不是"干活要用的家伙"。CAX 是研发里建模、仿真的作业工具，跟承载流程的统一平台一样，都是手段，只是角色不同：平台是"路"，CAX 是"车"。标准里能对上的两个词，一个是"基础设施"（组织运行所需的设施、设备和服务构成的系统，ISO 9000:2026 §3.4.3），一个是"资源"（过程执行中被利用或消耗的资产，15288:2023 §3.37）。

解析： 有人会误以为"CAX 不是活动，所以不属于运营体系"——这是把"运营"（活动）和"运营体系"（活动＋手段）混成了一个概念。手段是活动存在的前提：没有 CAX，现代研发运营根本转不起来。所以手段不是活动的附件，而是体系的一半。

示例： 〔案例·手段〕平台和工具不是一回事，但都在体系里：HDP 承载流程与数据，是"路"；CATIA/CAE 做建模与仿真，是"车"。没有 CAX，设计员小林连图都出不来——所以工具不是活动的附件，而是活动能发生的前提。

4. 把运营体系看作产品，其所有方、需求方、开发方、使用方分别是谁

答案： 所有方是企业治理层（董事会/最高决策层），它决定这套体系"为什么存在"，也批准大改；它干的就是治理三件事——指导、监督、担责（ISO/IEC 38500:2024 §3.3〔库外〕），对应组织里的最高管理者（ISO 9000:2026 §3.1.3）。需求方是企业里所有角色，市场、研发、生产、售后都会提要求，标准管他们叫利益相关方（ISO 9000:2026 §3.1.4；15288:2023 §3.44）。开发方是把要求做成方案并落地的那支队伍，标准角色是供应方（§3.45）。使用方是天天用它干活的员工，标准角色是用户（§3.53）。此外还有运维方，盯着绩效、催着改进。

解析： 五方（本课的分法）之所以有用，是因为它把"谁说了算、谁提要求、谁做、谁用、谁管"分开了。分开之后才好对上标准角色：治理→38500 §3.3；相关方→9000 §3.1.4 / 15288 §3.44；供应方→§3.45；用户→§3.53；获取方→§3.1。对上之后有个实际好处：出了问题知道该找哪一方，而不是笼统说"大家都有责任"。

示例： 〔案例·五方〕董事会定"做机载航电、三年不跨界"并批大改（所有方）；总经理李承志和各业务域提要求，主机客户代表王工、适航代表何工也提要求（需求方）；老张带的流程与数字化团队把要求做成方案并落地（开发方）；设计员小林、工艺员老吴、试验员小许、外场排故老黄天天在上面干活（使用方）；周经理带运营管理部盯指标、催改进（运维方）。

5. 运营体系是一个系统，运营体系的开发实现并运行应使用什么方法论

答案： 用系统工程那一套正向设计：先把需求弄清楚，再做架构与设计，然后实现，最后运行起来。落到运营体系就是三态——设计态出业务蓝图（一句话说清业务整体怎么转）以及由它展开的流程、制度、组织设计、指标、IT 需求；生产态把制度颁布、流程上线、系统建成；运行态让它天天转、按月量绩效。运行中发现不对，回头改设计，这就是一轮 PDCA。

解析： 正向设计是"从需求往前推"，不是"拿现成的改改"（逆向工程）。标准给的是过程链，从业务或任务分析（15288:2023 §6.4.1）一路到运行维护（§6.4.12–§6.4.13）。这里有个最容易讲错的地方：**架构与设计是两个并列过程**（§6.4.4 / §6.4.5），不是"架构粗、设计细"的两级。三态本身是本课的讲法，标准里没有这三个词。

示例： 〔案例·方法〕华翔没另起炉灶，用的是系统工程那一套：先把目的与需求弄清楚（PU-01、RQ-02），再定架构与设计（CP-01、DS-01~06），然后实现——制度颁布、流程上线、HDP 建成，最后转起来看指标。一轮走完不算完，运行中发现偏差就回设计态改，这是第二轮 PDCA。

二、产品与产品数据

1. 产品开发含系统及各构件过程，视为一组流水线，线上流动的是什么

答案： 流动的是产品数据（ISO 10303-1:2021〔库外〕）。把开发过程看成流水线，三样东西就清楚了：工件是产品数据（需求规格、设计模型、验证证据）；工位是需求定义、设计、实现、验证、移交；增值就是每一站往数据上加东西。区别只有一句：制造流水线流的是实物，开发流水线流的是数据。数据流到"定稿受控"那一刻，就成了一条基线（15288:2023 §3.8：配置项经正式批准、在特定时刻被正式指定并固定的版本）。

解析： 开发过程没有实物可摸，工件全是信息：需求数据流进设计，设计数据流进验证，验证证据流进基线，一路变成受控配置。标准侧还有两个词对得上：数据以"信息部件"的形态存在（§3.18：为供人和机器使用而生成、存储和交付的可单独识别的信息体）；冻结点就是基线（§3.8）。

示例： 〔案例·数据·工件〕把研发看成流水线：工位是需求、设计、试制、验证、移交；每站的工件都是产品数据——RQ-02、AD-01、DS-02、VR-01。数据流到评审通过、冻结成基线 BL-01，这一站才算走完。线上从来不流动实物，只流动数据。

2. 在产品开发过程中，各利益相关方所贡献的价值最终体现在什么载体上呢

答案： 落在产品数据上。所有相关方围着同一份数据读写，谁贡献了什么，最后都体现在这份数据里（产品数据定义见 ISO 10303-1:2021〔库外〕；利益相关方指对系统享有权利、份额、主张或利益的个人或组织，15288:2023 §3.44）。客户既是数据的源头，也是数据的归宿——要求从客户来，能不能兑现也要回到客户那里验。

解析： 内部相关方围着数据读写，客户则走四条路进来：提需要、使用、反馈、确认。所以协作的实质是数据读写，会议只是让数据对齐的形式。要让"谁的贡献落在哪份数据上"可查，靠的是可追溯性（§3.52：两个或多个逻辑实体之间可辨认的关联）。

示例： 〔案例·数据·载体〕主机客户王工提的交付要求、供应商回传的材料数据、外场保障老黄回填的故障数据，最后都落在同一套产品数据上：要求进 RQ-01，材料数据挂在 BOM 上，故障数据进 FRACAS。谁的贡献都得"落纸"，不落纸等于没贡献。

3. 产品与其产品数据是什么关系，开发产品本质上就是开发产品数据吗

答案： 产品有两个存在：一个是实物，一个是数据。产品数据就是"关于产品或产品族、能被机器或人读懂并处理的那份信息"（ISO 10303-1:2021〔库外〕；老版那句"与其整个生命周期相关"只作历史对照）。开发期实物还没生出来，产品只以数据形态活着，所以开发产品，本质上就是开发产品数据。

解析： 标准也是这个落点：设计和开发把要求变成"更详细的要求"（ISO 9000:2026 §3.3.11），而"规范"只是把要求写成文件（§3.8.8）；设计定义过程的目的，是"提供关于系统及其元素的足够详细的数据与信息"（15288:2023 §6.4.5.1）。数据细到能制造、能验证，开发才算完。边界也要说清：这话只在设计阶段成立，制造阶段有实物，所以是"开发 = 把定义生成出来"，不能说"产品就是数据"。

示例： 〔案例·数据·本体〕装机之前，适航审查要看的是产品数据包：需求基线、数字样机、验证证据；实物还停在装配线上。这就是"开发期产品以数据形态存在"——数据齐了、验过了，产品才算被定义出来。

4. 为什么应该用产品数据清单而非业务流程活动定义开发团队成员职责

答案： 因为职责的本质，就是"你负责交出哪几份数据"。用活动定义职责，只能说到"做了什么动作"；用数据定义职责，才说清"交出什么"。数据职责有三个好处：可较真（拿规定要求一条条对）、可挂机制（权限、签字、审批链、考核都能挂到这份数据上）、可度量（缺陷率、覆盖率拿数据说话）。缺了流程，数据的责任还在；缺了数据，流程的责任就空转。标准侧的支点是配置管理那几个词：配置项（§3.10.9）、配置信息（§3.10.8）、配置基线（§3.10.12），而"谁对哪个配置项负责"正是配置管理落到人头的方式（ISO 10007:2017〔库外〕）。

解析： 活动不好较真，也没有硬机制可挂；工件才是承接、验证、追责、考核的唯一锚点。工程上的成熟做法就是把归属写进数据本身——代码库里的 CODEOWNERS 就是"每份代码谁负责"，数据有主，缺陷就有去处。这也是 15288:2023 §3.52 可追溯性在人这一侧的具体兑现。

示例： 〔案例·数据·职责〕团队职责按数据清单写，不按活动写：小林负责 CP-01 与 AD-01，老吴负责 MF-01 工艺卡与 PBOM/MBOM，小许负责 VR-01 证据包，小赵负责 RQ-03 清单。出了缺陷查 owner，一看数据就定位到人；若只记"参加过几次评审"，永远查不清。

5. 判断管理是否精益，就看活动是否让产品数据增值，这样说对不对呢

答案： 大方向对，但要加三条限定（精益的价值判据：Womack & Jones 五原则、TPS 七类浪费）。第一条，价值由顾客定义，不是你说的算。第二条，"不贡献"不等于"浪费"——审批、合规、使能这三类不创造价值却省不掉，属于"必要非增值"，办法是压缩，不是砍掉。第三条，要同时看两面：内容是不是顾客要的，流动是不是有效率的。真正增值是让数据的内容或状态变好，而且这份好是顾客要的（比如评审把关、需求据此改准）。至于忙了一圈、数据一动没动（等审批、返工、重复劳动），就要追问值不值。

解析： 为什么加第三条限定？因为只看"数据有没有变"会走偏：加了一堆顾客不要的特性，数据也变了，那是过度加工。标准给了两句现成的话：过程被策划并在受控条件下实施，"以增加价值并确保能够达到预期的结果"（ISO 9000:2026 §3.3.1 注 4）；效率就是"所取得的结果与所用资源之间的关系"（§3.7.16），改进就是"提高绩效的活动"（§3.2.3）。

示例： 〔案例·数据·增值〕同一份 RQ-03：评审把"响应要快"改成"冷起动≤50ms"，数据内容真变了，是增值；适航合规检查让数据带上可追溯标注，属"必要非增值"，该压缩不能砍；而每周那个只通报进度、没有议决事项的例会，数据一动没动，是浪费。

三、产品需求与质量

1. 基于ISO标准，什么是需求、需要和期望，以及声明、隐含和强制的

答案： 需求，就是"明示的、通常隐含的、或必须履行的需要和期望"（ISO 9000:2026 §3.5.1；标准中文本写作"要求"，本讲沿用"需求"，一个词两个译名）。需要是硬诉求，缺了会痛；期望是软诉求，缺了会失望。剩下三个词说的是呈现方式：明示的 = 说出口、写进合同；通常隐含的 = 行业惯例默认，没人说但大家都这么认为；必须履行的 = 法规强制，无条件满足。三条并列，都是需求，没有主次。另一条口径来自系统工程：需求是"转化或表达需要及其相关约束和条件的陈述"（15288:2023 §3.36）。

解析： 两句标准正好互补：ISO 9000 回答"需求是什么"（三种形态的需要与期望），15288 回答"需求怎么表达"（能写成陈述，把约束和条件带上）。三形态最容易被误读成三选一——真正的意思是三样都得收：明示的漏了，验证没参照；隐含的漏了，是缺陷（标准注 1 讲了什么叫"通常隐含"）；强制的漏了，出局。

示例： 〔案例·需求·三形态〕华翔的航电设备：合同写"MTBF≥5000 小时"是明示；"高原机场不降额"是行业惯例默认的隐含；适航规章条款是必须履行。三种漏一种就出事——漏隐含，客户验收不认；漏强制，装不了机。

2. 什么是规定的需求，为什么要把隐含的和强制的需求转为规定的需求

答案： 规定需求，就是"写下来、明示出来的那份需求"（ISO 9000:2026 §3.5.1 注 2：规定的要求是明示的要求，例如写在成文信息中）。它是需求的子集，也是验证的参照——只有写成文的、具体的、能验的，才能设计出测试去证明。隐含的和强制的需求不转过来，验证（§3.11.12）就够不着：看不见，就验不着；验不着，质量就无从衡量。

解析： 隐含需求没人说出口，得主动去挖；强制需求躺在法规里，得摘进产品需求文档。这两类不进规定需求，验证照不到，缺陷还会往下游传。要注意标准写的是"例如写在成文信息中"——"例如"不是硬条件：口头要求也是明示的，但不成文就作不了采信依据。

示例： 〔案例·需求·规定化〕适航条款躺在规章里，得一条条摘成 RQ-03 里的规定需求（如"AS-041：高原 4500m 起动正常"）；客户那句"备件不能断档"也得转成"48 小时内补件"。没转成文的，验证就照不到，只能等外场出事。

3. 基于ISO标准，什么是质量和固有特性，质量定义中的需求是什么类型

答案： 质量，说白了就是"你自己的东西，把要求满足到了什么程度"——满足得多就高，满足得少就低（ISO 9000:2026 §3.5.2：客体的一组固有特性满足要求的程度）。这里的关键词是"固有特性"：它是东西自己长出来的本领，比如功能、性能、材质；不是外面贴上去的标签，比如价格、品牌（§3.10.1：特性是可区分的特征，可以是固有的，也可以是赋予的；§3.10.2：质量特性是与要求有关的固有特性）。所以质量这把尺子量的永远是东西本身。质量定义里的"要求"是广义需求（§3.5.1）——明示、隐含、强制三种都算，规定需求只是其中一部分。

解析： 定义宽、落点窄，这是这一问的要害。定义层面已经把"客户没说出口的要求"包进来了；可真要拿尺子量，只有写成文的规定需求才量得着。客户心里那句"要耐用"不写下来，你既验不了也判不了——这就是"需求必须显性化"的根子。三个条款别混：特性是上位概念，质量特性才与要求挂钩，质量用"一组固有特性的满足程度"来定义。

示例： 〔案例·质量·固有〕显示延迟、功耗、抗振是设备自己的固有特性；"比竞品便宜"是贴上去的赋予特性，不进质量这把尺子。质量量的是"固有特性满足要求到什么程度"——延迟要求 60ms、实测 48ms，才叫满足。

4. 好需求的判据有哪些，判据是怎么得来的，为何对产品开发非常重要

答案： 判据分两层。第一层门槛：这份需求整体能不能当设计输入？四条——完整（类别没缺口）、无歧义（一条只有一种理解）、不冲突（条目不打架）、隐含需求已识别（没说出口的也收齐）（ISO 9001:2015 §8.3.3〔库外〕）。第二层看单条好不好，十一条特性——必要、恰当、无歧义、完整、单一、可实现、可验证、正确、符合、一致、可追溯（ISO/IEC/IEEE 29148:2018〔库外〕）。逐条的意思很直白：删了它系统就少一份能力（必要）；只说做什么不说怎么做（恰当）；一句话只说一件事（单一）；在成本进度技术约束下做得出来（可实现）；有办法证明满足了（可验证）；能一路追到上级要求和实现验证（可追溯）。

解析： 判据不是凭空立的，是反推出来的：设计和开发就是"把要求变详细"（ISO 9000:2026 §3.3.11），四条门槛有一条不过关，这个转化就不可靠，所以 §8.3.3 只是把前提写成了条款；29148 那十一条是踩坑之后的反写——出过歧义返工，就立"无歧义"。标准还给了两个硬要求：需求要写成带约束和条件的陈述（15288:2023 §3.36）；需求到系统元素的关联要可追溯（§3.52）——所以"可追溯"不是软要求。

示例： 〔案例·需求·判据〕"响应要快"进了需求库，设计按 30ms 做、验证按 200ms 测，验收时对不上、返工——缺的正是"无歧义"。改成"冷起动≤50ms"，设计、验证、验收读的是同一把尺子；再补上到 RQ-01 的追溯，这条需求才算合格。

5. 质量与需求的关系是什么，当质量与进度发生冲突时，该怎么处理呢

答案： 需求是标尺，质量是"满足标尺的程度"（ISO 9000:2026 §3.5.2）。所以底线是满足需求——要砍就砍需求，别砍质量。难点在于：进度本身也是一条需求（ISO 9001:2015 §8.2.2〔库外〕：交付时间和交付条件同样是顾客要求的一部分）。所谓"质量与进度冲突"，本质是同一套需求里几条在打架：能砍的是非关键、过度设计的那几条，不能动的是必须履行的和关键特性。

解析： 处理顺序有两步。先判真假：估算失真造成的假冲突，改计划就行，不动质量。真冲突就走需求谈判加正式变更（ISO 9001:2015 §8.3.6〔库外〕），以需求判据为锚：需求满足是质量底线，进度是过程约束；砍需求必须经利益相关方协商、变更留痕，不能单方砍。标准里还有个提醒：合格 = 满足要求（ISO 9000:2026 §3.5.9），不合格 = 未满足要求（§3.5.13）——砍需求就是移动合格线，必须留痕。

示例： 〔案例·质量·冲突〕型号节点压过来，有人提议把"高原起动"这条适航条款当可砍项——这是动红线。正道是把非关键的性能裕量剪一剪、走 CR-01 变更评审、让客户代表和适航代表签字确认：进度可以谈，合格线不能私自移。

四、何为设计与开发

1. 基于ISO标准，何为设计与开发，对该定义中的主要元素，你如何理解

答案： 设计和开发，就是把"对这个东西的要求"变成"对它更详细的要求"的一组过程（ISO 9000:2026 §3.3.11）。拆开看六个元素：①客体——被设计的东西，不限于产品；②要求——输入得是要求，不是点子；③转化——输入输出必须不一样，一样就不叫设计开发；④更详细——这是唯一的判据，要更具体、更可做、更可验；⑤一组过程——不是一个动作；⑥递归——一个项目里可以有好几轮，系统、子系统、构件各一轮（注 1）。还有一句要记住：标准明说"设计""开发"两个词有时同义、有时指整个设计和开发的不同阶段（注 2），二者不是包含关系。

解析： 这套定义落到下游就是"每条要求都细化到别人照着能做"。标准没有规定递归几层，只说"一个项目中可以有若干个设计和开发阶段"——所以分层是工程决定的，不是标准规定的。工程侧再补一个词就更完整：设计（名词）是"对系统元素及其关系的规定，完整到足以支撑架构的合规实现"（15288:2023 §3.13）——这就是设计开发过程的产物长相。

示例： 〔案例·设计·定义〕客户说"要能在高空低温下工作"，设计开发把它变成"-40℃、连续 72 小时、误差≤±0.5℃"的系统级规格——更具体、更可做、更可验，输入输出确实不一样。这条规格再往下细化成温控部件的要求，一轮一轮往下走。

2. 产品设计过程起点终点各有什么产品数据，设计的终点是什么的起点

答案： 起点是规定需求（把利益相关方的需求规范化、写清楚的那一份）；终点是可实现的方案。实物产品通常是八个方案：技术、物料采购、制造（工艺）、测试、交付、部署、操作、运维（本课按下游环节展开的讲法，不是标准清单）。起点和终点本质都是要求，差别只在详细程度——到终点，下游每个环节都能照着做。标准侧可以对齐两条：设计输出要满足后续产品和服务提供过程的需要，方案可以按下游环节增删（ISO 9001:2015 §8.3.5〔库外〕）；设计定义过程的目的是"提供足够详细的数据与信息，以按照系统需求与架构实现解"（15288:2023 §6.4.5.1）。设计的终点是生产的起点——最后一个方案补齐，才算完，质量也才被完整定义。

解析： 八个方案各自的用处很实在：技术方案证明原理可行；物料采购方案给出采购依据；制造方案就是工艺，支持制造；测试方案支持测试；交付、部署、操作、运维四个方案，分别让产品能交出去、装起来、被用起来、被维护。处置另有 15288:2023 §6.4.14 处置过程，不硬塞进这八个方案里。

示例： 〔案例·设计·终点〕研发运营体系的设计，起点是 RQ-02（规范化的需求），终点是八方案齐备：技术（TE-01 一体化平台方案）、采购（PR-01 算力与软件许可）、制造/工艺（MF-01 流程建模与配置规范）、测试（TS-01）、交付（DL-01 分批上线）、部署（DP-01 环境与权限）、操作（OP-01 岗位手册）、运维（MT-01 平台运维与变更）。第八个方案落地那天，才是生产的起点。

3. 产品工艺编制是否属于产品开发，为什么，它输出的产品数据是什么

答案： 就活动的性质说，属于。标准允许给"设计和开发"加限定语，说明被设计的是什么——产品、服务，或者过程（ISO 9000:2026 §3.3.11 注 3），工艺就是"过程（制造）设计和开发"。但换个账本看，它的产出登记在"实现过程"名下（15288:2023 §6.4.7）——性质和归属是两把尺子，别用一把去量另一把。工艺编制干的事，是把"构件做成什么样"的详细要求，翻成"制造环节怎么做"的可实现要求；它是八个方案

里的制造方案（工艺）之一。输出的产品数据就是制造方案：工艺规程、工艺卡，也就是 PBOM/MBOM（试制 / 量产的工艺物料清单）。

解析： 产品开发不只是画图写代码，还包括构件怎么做、怎么装、怎么检的技术开发。缺了工艺，产线上没人知道"怎么装、用什么工装、按什么标准检"，设计就还停在"可实现"的门外。要注意"属于"说的是活动性质（ISO 9000 承认"过程设计和开发"），不等于它在 15288 的过程清单里占"设计定义过程"的位置——两套标准各管一件事。

示例： 〔案例·设计·工艺〕设计说"流程要在平台上跑、数据一次录入"，工艺员老吴把它翻成能施工的东西：《流程建模与配置规范》＋配置清单——谁建、按什么命名、哪些字段必填、怎么校验。这就是"过程（制造）设计和开发"；产出登记在实现过程名下，但它确确实实是设计的一部分。

4. 设计开发、需求、质量三个概念，它们彼此之间究竟是怎样的关系呢

答案： 三句话就能串起来：需求是输入也是判据；设计开发是把要求变详细的过程；质量是拿要求去量结果。所以链条是——需求定标尺，设计开发把标尺刻细，质量照着标尺读数。逐级递归就是这个意思：这一轮输出的更详细要求，下一轮又成了判据。

解析： 三者首尾相接：需求说"要什么"，设计开发把"要什么"翻译成"做到什么程度"（§3.3.11），质量问"做到了没有"（§3.5.2）。为什么质量要到终点才完整？因为固有特性要等八方案齐备、产品被完整定义，才全部规定下来。还有一条因果：改了需求就是改了标尺，设计开发得重转，质量判定也跟着变。标准依据就三条定义——需求 §3.5.1、设计开发 §3.3.11、质量 §3.5.2，恰好构成"输入—过程—参照"。

示例： 〔案例·设计·关系〕一条链看三样：RQ-03 里"告警 30 秒内上报"是判据；设计开发把它变成显示逻辑、告警优先级、接口时序等更细的要求；等八个方案齐备，这些要求全部落成"固有特性"，质量才完整可量。改了需求就是改了标尺，设计和判定都得重来。

5. 运营体系开发过程的终点是什么，它输出的产品数据具体包括哪些呢

答案： 终点是设计态齐备——按设计开发的口径（ISO 9000:2026 §3.3.11），把运营体系的要求细化到下游每个环节都能照做，也就是"足够详细的数据与信息"（15288:2023 §6.4.5.1）。设计态齐备了，就是生产态（IT 实现）的起点；再往后进运行态（IT 运行），运行再回头反馈设计。输出的产品数据是一套运营方案文档：业务蓝图（业务整体怎么转的总纲，其余各项由它展开）、流程文件、制度文本、组织设计、指标体系、IT 需求——六项（本课的清单）。

解析： 只写制度、只建系统，都只走到设计态或生产态；体系得在运行态真转起来才算建成。设计态不齐备，IT 那边无从开工：制度、流程、指标、IT 需求都得细到 IT 能开发。还有一处边界要划清：IT 系统拿到 IT 需求之后的方案设计，属于 IT 系统这个"手段"自身的设计态——手段有它自己的三态，不混进运营体系的三态里。还要说清一件事：这六项是业务侧的设计交付物；要让它们在平台上真跑起来，还得配齐八个实现方案（TE-01…MT-01，见本组第 2 题）——**六项定义"做什么"，八方案回答"怎么落地"**，两者不是两套终点。

示例： 〔案例·设计·运营体系〕研发运营体系的设计态什么时候算完？六份文档齐备：业务蓝图 DS-01（一句话说清业务怎么转）、流程文件集 DS-02、制度文本 DS-03、组织设计 DS-04、指标体系 DS-05、IT 需求规格 DS-06。这六份不齐，IT 那边不能开工——所以它们是生产态的入场券。

五、系统工程之工程

1. 系统工程、软件工程、硬件工程语境中，什么是工程，怎么理解X工程

答案： 工程 = 拿科学原理加人的判断，在约束下用系统方法，造出有用又安全的东西（这是 ECPD 1941、ABET 1979、INCOSE 2019、IEEE 610.12〔库外〕四家定义收敛出来的共同内核）。它跟四样东西划清界限：跟科学不同，科学求真、工程求能用；跟手艺不同，手艺靠个人手感，换人就断；跟艺术不同，艺术为自己表达，工程为别人能用。这里的"系统方法"是一条链：需求→架构与设计→实现→验证→维护。所谓 X 工程，就是把工程用到 X 上：软件工程是软件的生存周期，硬件工程是物理部件的生存周期，系统工程是复杂系统的整体（它整合部件、自己不造部件）。三者骨架一样、载体不同，所以 X 工程 = X 生存周期工程（生存周期 = 从构想到退役的演化，15288:2023 §3.21）。

解析： 有个细节值得讲：IEEE 610.12 定义的是软件工程——"把工程应用于软件"，它把"工程"当已知概念引用了，并没有定义工程。系统工程的定义见 15288:2023 §3.50（源自 INCOSE），那句话是"一种跨学科、综合性的途径……使工程化系统得以成功实现、使用和退役"——里面根本没出现"工程"这个词。两个定义都覆盖生存周期，这才是"X 工程 = X 生存周期工程"的由来。（链的末端这里作"维护"，运营语境常说"运行"——15288 里运行与维护本来就是两个过程：§6.4.12 / §6.4.13。）

示例： 〔案例·工程·X工程〕华翔的"软件开发"不是写代码那一段：需求→架构与设计→编码→验证→上线运维，整条链换个人也能交付，才叫软件工程。研发运营体系的开发同理，也是一条生存周期——把制度、流程、平台做成体系，不是谁拍脑袋写几份文件。

2. 工程的底座是什么，工程的特征有哪些，它与手艺的区别究竟是什么

答案： 底座是科学原理加判断与创造——工程是应用原理，不是发现原理，物理定律、数学模型、算法理论都是依托；但只有原理不够，还得有人做判断、做取舍（ECPD 与 ABET 定义里那句"以判断力应用数理知识"说的就是这一半）。过程特征有三条：systematic 系统化（有章法）、disciplined 有纪律（守规矩）、quantifiable 可度量（可验证）——这组形容词出自 IEEE 610.12〔库外〕对软件工程的定义。与手艺的区别只有一个词：可复现。手艺靠个人经验和技巧，结果跟着人走；工程靠流程、标准、文档、评审、测试，换个人也能交付。试金石就一句："你不在，交付还维持得住吗？"维持得住是工程，维持不住是手艺。

解析： 三要素的原文是 systematic / disciplined / quantifiable，所以"有纪律"对应 disciplined（规范），"可验证"对应 quantifiable（可度量）——顺序别背反。手艺不是贬义，差别只在"换个人能不能交出同等质量"。标准侧给了一个组织机制：识别、监视并管理自己的过程及其相互作用（过程方法，ISO 9000:2026 §3.3.4）——这就是"换人也能交付"靠什么撑住。

示例： 〔案例·工程·底座〕试验室老主任的方案全在脑子里，他休假两周，同样的试验没人复现得出来——这是手艺。小许把它写成试验规程、作业指导书和记录台账，换个人照章也能出同样的结果，这才是工程：靠原理与判断，更靠可复现的过程。

3. 什么是工程化、系统化，系统化、工程化、流程化、信息化四者关系

答案： 系统化就是"做事有章法"（systematic），四个特征：有结构（拆成清晰步骤）、有章法（每步有标准和检查点，但给人留判断）、可重复（换人结果还是稳的）、全覆盖（从需求到退役不落环节）。工程化 = 系统化 + 有纪律 + 可度量。四者的顺序是递进：系统化（方法地基）→工程化（加纪律和验证）→

流程化（把做法固化下来）→信息化（让系统承载）（这是本课的讲法）。最反直觉的一条：流程化不等于工程化——有流程不代表有工程，工程化还必须守纪律、可验证。

解析： 系统化还要跟两个词分清：一是机械化（把每步做成算法），工程有章法但每步仍给人留判断，像乐队而不像机器；二是 systemic（看问题有全局），systematic 管"怎么做对"，systemic 管"系统对不对"。标准侧的对应件是过程与过程方法（ISO 9000:2026 §3.3.1、§3.3.4）：把活动组织成"相互关联或相互作用、利用或转换输入、交付结果"的一条条过程，就是系统化落地的样子。

示例： 〔案例·工程·四化〕工单录入本来就有流程，可录完没人核对、出错查不到记录——这是"流程化"，不是"工程化"。补上"录入→核对→抽检"的标准与检查点、结果进台账、换人也能稳交付，才算工程化；再往下把流程固化、让平台承载，才走到信息化。

4. 个人能力和个人工程能力是一回事吗，那团队能力和团队工程能力呢

答案： 不是一回事。个人能力是"我自己能干"；个人工程能力是"我干出来的东西可复现，换个人也能接住"。能力再强、只要复制不出去，那还是手艺。团队同理：团队能干，不等于团队有工程能力；买了工具，也不等于有了工程能力。判据还是那块试金石：个人版问"我明天走人，团队还能交出同等质量吗"，团队版问"最值钱的那个人走了，交付还维持得住吗"——能，就是有工程能力。

解析： 差别在"可复制性"：工程能力是把个人或团队的本事，变成可教的规则、可查的文档、可检验的标准，从而变成组织的能力。工具（CI/CD、测试平台）不会自动带来工程能力——一套只有个别人会敲的命令，跑起来只是换了壳的手艺。标准侧有两个词别混：capability 是客体实现输出、满足输出要求的能力（ISO 9000:2026 §3.5.10）；competence 是人员运用知识和技能取得预期结果的本领（§3.10.6）——中文都译"能力"，但英文两个词，不能互推。工程能力要落在过程与准则上，不落在个别人的本领上。

示例： 〔案例·工程·能力〕HDP 上线了，发版却靠只有小唐会敲的三条命令、李四记得的参数表——平台是工具，真本事锁在个人脑子里，小唐一休假发布就停。把命令和参数表写进发布规程与文档、换个人照章也能发版，团队才算有工程能力。

5. 什么是工程师，什么是流程工程师，二者能力清单和职责清单是什么

答案： 工程师不是一个头衔，而是一组本事：能掌握和应用科学原理，能在约束下权衡取舍，能用系统方法把全过程组织起来，能造出有用且安全的东西；头衔、证书、年限都不定义工程师。能力有六项——科学素养、方法论素养、实践能力、职业伦理、工程判断、可复现意识（本课的清单，与 ABET 毕业生成果交叉验证）；职责是对交付物（产品数据）和方案后果负责。流程工程师把这套本事用在运营体系上：把运营体系当产品来开发，技能是六项能力再加领域五项——业务分析与需求转化、流程设计、全链贯通、运营体系产品数据、治理与改进意识；职责同工程师。

解析： 六项能力是从"工程四层"（原理、判断、约束、方法）推出来的，再拿去跟 ABET 成果七条对：沟通、团队这两条没被六项显式覆盖，所以只算交叉验证、不算等价。职业伦理那一条，对应 NSPE 伦理准则"保护公众健康与福祉"。运营体系在这里 = 运营活动 + 支撑运营的手段（第 1 组第 1 题的合成定义）；放到标准语言里，最接近的组织侧概念是管理体系（ISO 9000:2026 §3.4.2：组织中一组相互关联或相互作用的要素，用以确立方针和目标，以及实现这些目标的过程）。

示例： 〔案例·工程·角色〕老张是流程工程师，没有工程师职称：他能把"航材 48 小时到位"这个要求做成维修流程、备件台账和派单规则，可靠、可复现、可追溯——这就是工程师该有的六项本事（科学素养、方法论、实践、职业伦理、工程判断、可复现意识）再加运营领域的五项。头衔不定义工程师，交付物定义。

六、目的目标与需求

1. 什么是系统的目的，给出定义中主要元素，目的对系统开发有何意义

答案： 目的是回答"这套系统为什么被研制、为什么存在"的一句话，它是价值陈述，可以虚、不必量化。主要元素有四：①存在理由——在价值层回答"为什么做"；②链条地位——站最上游，目标和需求都从它展开；③来源——由治理机构在利益相关方的关切上确立、经业务分析讲明白，业务方签字确认（落到《系统目的或任务理解确认单》：原文引用 + 复述理解 + 签字），开发团队不拍板；④归属——属组织层，区别于某个用户自己的使用目的。意义就是定方向：它是需求与目标的源头、质量的锚点、评审和确认的参照。方向没锚住，就变成"正确地做错事"。

解析： ISO 9000:2026 里没有"目的（purpose）"这个独立条目，但有两处可用：§3.4.11 的使命（mission）就说"由最高管理者表述的组织存在的目的"；§3.7.11 注 3 也容许把目标表述成"目的"。组织层的宗旨定义见 ISO 37000:2021 §3.2.10〔库外〕。系统层的标准落点是 15288:2023 §6.4.1 业务或任务分析——它定义总体战略问题或机会、表征解空间、确定候选解类；"定义任务、业务或运行问题"是它的活动之一，不在它的预期结果里。定义权在治理机构，不在利益相关方（38500:2024 §3.3〔库外〕：治理机构定方向并担责）。

示例： 〔案例·目的·定义〕华翔为什么要搞这套研发运营体系？不是为了"上个平台"，而是为了让机队任务不中断——型号交付稳、外场少停飞。这句价值陈述不量化，却决定了后面所有取舍；董事会把它写进 PU-01，业务方签字确认。

2. 什么是系统的目标，给出定义中主要元素，目标对系统开发有何意义

答案： 目标是"要实现的结果"（ISO 9000:2026 §3.7.11）。放到系统上说，就是系统预期达成、可以量的那个结果，是目的的可度量投影。四个元素：①可度量——要量化、能验收（"可度量"是本课的口径，标准原文只说"要实现的结果"）；②主语是结果——可以落在组织、机队、产品、服务、过程哪一层都行（注 2），不必是系统自己的属性；③由目的展开——给目的加上对象和时限，它就可衡量了；④一对多——一个目的可以展开成一组目标，各管一个切面。意义是把方向翻译成能验收的数：它是需求转化的起点、质量的锚点、评审确认的参照。不立目标，目的就只是口号。

解析： 标准注 2 允许目标对准整个组织，也可以对准某个项目、产品、服务、过程——所以"机队任务完好率"这种落在机队层上的目标成立。注 3 还有一层提醒：目标可以用"预期的结果、目的、运行准则、质量目标"等不同说法表达，所以"目的"和"目标"在口头语言里能互换，在链条上不能互换——目的定方向，目标必须能量。验收那一步归确认（§3.11.14）。

示例： 〔案例·目标·定义〕目的往下落成可验收的数：机队任务完好率≥95%、型号交付准时率≥98%、外场排故 4 小时内闭环。目标落在机队层、交付层，不一定落在研发运营体系自己身上；每月数一遍，够不够一眼就知道。

3. 系统的目的、目标和需求是什么关系，请你以一个具体产品举例说明

答案： 三者是三级递进：目的是方向（为什么，可以虚），目标是由目的展开的结果（是什么，必须能量），需求是由目标转化的判据（做到什么规格，必须可验）。要特别注意：目标→需求是转化，不是分解——它跨了类别，语义不会原封不动地保留下来（比如"任务完好率 95%"不能切成两条需求，得靠工程判断去导出）。链条反过来是可以追的：任取一条需求，都能问回它上面那个目标，再问回目的。

解析： 三者维度不同：价值、结果、规格。标准定义在 ISO 9000:2026 §3.7.11（目标）和 §3.5.1（需求）；"目的"没有独立条目，它的意思散见于 §3.4.11 使命和目标注 3，组织层则见 37000:2021 §3.2.10〔库外〕。系统工程把这条链落到了过程上：目的与任务在业务或任务分析里明确（15288:2023 §6.4.1），利益相关方需要与需求随后定义（§6.4.2），系统需求再往下定义（§6.4.3）——过程分三段，正是"转化不守恒"的机制解释。

示例： 〔案例·链条〕从"任务完好率≥95%"导出两条需求：MTBF≥5000 小时、故障告警 30 秒内上报。为什么是这两条？少坏、快修，两条都在撑那个数；反过来，任取一条需求，也能一路问回目标、问回目的——这就是可追溯。

4. 系统的目的是谁定义的，没有或不了解目的，系统开发会有什么问题

答案： 目的由治理机构在利益相关方的关切上确立、经业务分析讲明白；它不是天然长在那儿的，开发团队也没有拍板权（治理依据 ISO/IEC 38500:2024 §3.3〔库外〕；过程依据 15288:2023 §6.4.1，执行主体是组织层）。落到运营体系：定义权在治理层和经营层，业务方签字确认，开发者（含流程工程师）只承接、不定义。没有目的或不懂目的，就是"正确地做错事"：方向没锚，团队自己假定一个（往往是错的）目的，然后高效地推进，把本来不该做的系统做得尽善尽美，越努力偏得越远。

解析： 运营体系的目的是战略落地那座桥。15288 只规定了该过程的目的和结果（界定问题或机会空间、表征解空间、选定优选解类），并没有把定义权交给利益相关方——定义权在组织层，所以"开发团队不拍板"。架构师和流程工程师有质询权，没有改写权；这和治理分工是一回事：治理机构定方向并担责，管理层在方向内执行。

示例： 〔案例·目的·归属〕PU-01 上那条目的是董事会确认的，不是老张写的：他在评审会上只能问"这条目的和战略对得上吗"，不能改成自己想要的样子。业务方签字、开发方承接——这条界线划清了，后面才不会"做着做着换了方向"。

5. 开发运营体系，流程工程师应该最先明确它的目的还是目标，为什么

答案： 先明确目的。目的在最上游，定的是"对不对"；目标只是它的可度量投影——方向没锚住，目标根本无从操作化。但"明确"不等于"定义"：目的由治理层、经营层定义，业务方签字，架构师和流程工程师都没有定义权。分工是这样：架构师明确目的，是为了判断"什么才算基本"、定架构的形状；流程工程师明确目的，是为了设计真正支撑目的的流程。两人都要先过目的关，再帮业务方把目的立成可度量的目标。

解析： 运营体系的目的就是战略落地那座桥，它是流程存在与设计的判据；不知道目的，结果就是"正确地做错事"。标准侧有两条依据：架构的核心是"基本概念或属性"，而"基本"由目的界定（42010:2022 §3.2 里的"在其环境中"，配合 §3.12 所关注实体）；架构评估要判断"架构实体是否满足其预期目的"（42030:2020 §3.4）——目的因而是架构和评估共用的那把尺。

示例： 〔案例·目的·先后〕老张开工前先看 PU-01，再决定流程怎么画：目的是"少停飞"，那外场响应就得前移到客户现场，而不是把内部审批做得更漂亮。反过来，若先定"流程上线率 100%"这种手段指标，流程画得再精致，对机队任务也没有支撑。

七、评审确认与验证

1. 基于ISO标准，什么是评审，定义中有哪些主要元素，怎样理解它们

答案： 评审，就是"抬头看看这东西对不对路"：为了达成既定目标，对客体的适宜性、充分性、有效性做一个判断（ISO 9000:2026 §3.11.2；"确定"本身是查明一项或多项特性及其特性值的活动，§3.11.1）。元素有五：①客体——被评的东西；②既定目标——评审的参照是目标，不是需求清单；③确定——是判断动作，靠判断力，不要求客观证据，这一点跟验证、确认的"证实"是两种动作；④三问判据——适宜（对症吗）、充分（够全吗）、有效（管用吗）；⑤按注 1，评审也可以包括对效率（§3.7.16）的确定。

解析： "确定"和"证实"不是一回事：判靠判断力，证靠客观证据。抬头看方向的眼睛也不止评审一双，还有确认——评审在工作产品阶段先拦一道，确认在实物投入使用前再拦一道。没留下记录的评审不算评审，最多算聊聊天（IEEE 1028-2008〔库外〕讲系统性评审有三属性：团队参与、结果文档化、过程文档化）。"有效"这个词也有标准定义：策划的活动得以实现、策划的结果得以达成的程度（§3.7.17）——这就是"评审要看目标有没有真被达成"的条款依据。

示例： 〔案例·评审〕RV-01 那场评审，对着需求逐条都对得上，可一抬头看目标"任务完好率≥95%"——方案只堆了单机可靠性，没设计故障告警与降级策略，坏了照样全停。评审对的是目标，不是需求清单，这一关只有人抬头才看得见。

2. 基于ISO标准，什么是确认，定义中有哪些主要元素，怎样理解它们

答案： 确认，就是"拿到真实场景里，证明它真能用"：通过提供客观证据，证实针对特定预期用途或应用的那些要求已经满足（ISO 9000:2026 §3.11.14）。三个元素：①参照是预期用途或应用的要求——参照长在场景里，由目标用户、使用场景和任务、环境条件三样构成；②客观证据——得拿得出证据（注 1：试验结果，或其他确定形式的结果）；③证实——下"已满足"的结论。注 3 还允许使用条件真实或模拟。它和验证用的是同一套三要素，唯一的区别就在"参照是什么"。

解析： 预期用途的三样构成，正好对上 UAT 的"真实用户 + 真实任务 + 真实场景"。参照在场景里、不在文档里，而隐含形态天然算在要求内（§3.5.1 注 1 讲的"通常隐含"），所以确认能兜住验证够不着的隐含需求。系统工程侧讲的是同一件事：确认过程的目的是提供客观证据，证明系统在预期运行环境中满足业务或任务目标以及利益相关方的需要与需求（15288:2023 §6.4.11.1）；标准还把利益相关方需求界定为"对由此产生的每项运行能力进行确认所依据的参照"（§6.4.2.1）——所以参照是使用，不是条目。

示例： 〔案例·确认〕需求清单全过了、测试也过了，可真实机队"装订—飞行—回收"跑下来，高原振动时告警延迟到 40 秒——这条"振动时告警也要及时"的预期用途要求，清单上从来没写过。不是没做对，是没被识别；只有拿到真实场景里跑，才兜得住。

3. 基于ISO标准，什么是验证，定义中有哪些主要元素，怎样理解它们

答案： 验证，就是"照着自己写下的要求，一条条证明做到了"：通过提供客观证据，证实规定的要求已得到满足（ISO 9000:2026 §3.11.12）。三个元素：①参照是规定的要求——"规定"就是"被明示"，口头说的也算明示，但要拿来当验证参照，必须成文、可采信；②客观证据——标准注 1 列的是检验（§3.11.11）的结果，或其他确定形式的结果（比如替代计算、评审文件）；试验结果在确认的注 1 里（§3.11.13、§3.11.14）；③证实——要对着全部规定需求逐条下结论，缺一条都不算验证完成。系统工程侧是同一件事：验证过程证明系统满足其系统需求与架构（15288:2023 §6.4.9.1）。

解析： 口头要求也是"明示的"，但不成文就作不了采信依据——没有成文的规定需求，就等于没有验证的尺子。客观证据不等于"测试"：测试只是取证手段之一，检验结果、替代计算、评审文件同样能当证据。证实则要求逐条下结论，"跑完了"不算结论。再记一句因果：尺子在文档里，够不着隐含需求——这正是确认不能省的理由。

示例： 〔案例·验证〕测试通过率 96% 看着漂亮，可剩下 4% 的需求，验证方案里压根没排活动、一条证据都没有。验证是对全部规定需求逐条下结论，不是算百分比；缺一条证据，整份"验证通过"都不能签。

4. 目的、目标、需求与评审、确认、验证什么关系，确认的依据是什么

答案： 源头是一条链：目的展开成目标（ISO 9000:2026 §3.7.11），目标转化出需求（§3.5.1），需求明示成文就成了规定需求。链上站三个看门人，各拿各的参照：评审拿既定目标，验证拿规定需求，确认拿"为特定预期用途或应用的那些要求"。所以确认的依据就是预期用途本身；再往上，确认还能追到业务或任务目标以及利益相关方的需要（15288:2023 §6.4.11.1），而评审就止于目标这一层。

解析： 三个看门人各守一种错：评审守"方向错"（目标选错了），验证守"转写错"（需求转错了），确认守"匹配错"（真实用途没兜住）。证据强度也是分层的：评审只要"确定"（§3.11.1），验证和确认都要求"通过提供客观证据证实"（§3.11.12 / §3.11.14）——所以"没记录的评审不算评审"这句话有条款依据。

示例： 〔案例·三参照〕三把尺子各管一段：评审 RV-01 拿目标量（这方案管用吗）、验证拿规定需求逐条对（证据进 VR-01）、确认 VA-01 拿真实用途量（飞行员在高原用得住吗）。三份记录凑齐，才敢说这套体系"对、全、能用"。

5. 运营体系某业务系统上线运行但未解决业务问题，可能缺哪一道关口

答案： 缺的是评审——对着目标那一道关。"上线运行"只说明系统建成、在跑；"没解决业务问题"说的是另一个层面：系统没实现"解决业务问题"这个目标，目标（可度量的结果）没达成。守目标的是评审：评审缺位，设计阶段就没人问一句"这套系统对达成目标真的有效吗"，做出来自然问题依旧。

解析： 三个看门人各守一种错，先分清是哪一种：目标选错、或方案跟目标脱节——缺评审；目标没错、但真实用途和隐含需要没兜住——缺确认（只做验证不做确认，隐含需求会系统性漏失，就是"测试全过、上线没人用"）。"没解决业务问题"如果是目标本身就选错了，那正是评审"有效性"（§3.7.17）失守：评审是工作产品阶段抬头看目标的那道关，缺了它，问题只能等到运转以后才暴露。

示例： 〔案例·关口〕外场排故知识库上线三个月，排故时长没降——因为需求写成了"支持库存查询"，跟"排故快"这个目标不搭。系统在跑、数据也有，缺的是评审那一关：没人对着目标问一句"这套东西真能解决排故慢吗"。

八、系统架构与视图

1. 基于ISO标准，什么是架构，定义中有哪些要点，怎样理解它们

答案： 架构，说白了就两样东西：这个东西身上最要紧、绕不开的那几条概念和属性；再加上管着它怎么造、怎么改的那几条原则（ISO/IEC/IEEE 42010:2022 §3.2：实体在其环境中的基本概念或属性，以及为实现和演化该实体及其相关生存周期过程的管控原则）。要点有五：①被架的那个"东西"叫所关注实体（§3.12：架构描述的主体），不一定是系统——企业、组织、过程、业务、数据、产品都可以有自己的架构；②"基本"是说：去掉它，这个东西就不再是它、或者达不成它的目的；③"概念或属性"中间是"或"，不是"与"，从概念切入或从属性切入都行，不必两样齐备；④"在其环境中"——环境（§3.13）决定什么算基本；⑤管控原则是定义的一半，它管的是这个东西怎么实现、怎么演化，不是可有可无的补充说明。

解析： 最容易搞混的三样东西：架构、架构描述、框图。图是架构描述，架构描述是"用来表达架构的工作产品"（§3.3），而架构是那个抽象的东西本身——所以"实现可以换、本质不能换"，图会过时、那个决定还活着。还有一个常被误读的地方：元素和关系是架构**可以关乎**的对象（§5.2.2：架构能涉及构成元素、元素之间的交互、与环境的交互、行为与结构、以及管控其设计使用运行演进的原则），它们不在定义句里。架构工作（architecting，§3.1）也有明确定义：在实体整个生存周期里，对架构进行构想、定义、表达、编制文档、沟通、证实其正确实施、维护和改进。

示例： 〔案例·架构·定义〕华翔研发运营体系的架构，说白了就两样：几条绕不开的基本概念——研发业务一体化、数据一次录入全程复用、平台可配置不写死；再几条管控原则——流程变更必须一次评审、数据不许落地转抄。这些写进 CP-01 与 GP-01，而不是那张四层框图。

2. 在架构的定义中，"基本"的判据是什么，"基本"与系统目的有何关系

答案： "基本"的判据只有一条：对这个实体、在它的环境里、针对它的目的，是否必要且充分（这句话出自 IEV 831-01-01 Note 2，42010:2022 正文没有引它，属于二手说法，只能当佐证）。平时用的"去掉不是它 / 缺了垮掉 / 换了变质"三问，是这条判据的操作化：不可再分、不可缺失、不可替代——三问的落脚点是同一个，就是"它还在不在为这个目的服务"。再深一层说，"基本"是演化中的不变量：重实现、换实例的时候必须守住不变，否则这个东西就变成了另一个东西——因为人造的东西"还是不是它自己"，由目的决定，不由材料决定。

解析： "基本"不是东西的固有属性，它是个关系判断，所以每次都得补全：对哪个实体、在什么环境、针对什么目的。目的决定"基本"，"基本"又决定架构长什么样——可以说架构就是目的的形状化。于是有了一条经验：**目的漂移是架构失效的第一病因**——技术老了但目的没变，架构可能还健康；目的悄悄变了而架构没跟上，才是"遗留系统之痛"的根。标准侧也有印证：架构评估要判的一件事，就是"架构实体是否满足其预期目的"（42030:2020 §3.4）。

示例： 〔案例·架构·基本〕外场故障数据实时回传，在"少停飞"这个目的下是基本能力（缺了，排故只能等飞机落地）；可目的换成"核算维修工时"以后，它就成了实现细节。同一个功能是不是"基本"，由目的说了算，不由技术先进程度说了算。

3. 什么是架构的视图，TOGAF的四个架构，它们与企业架构是什么关系

答案： 架构视图是架构描述的一部分，是一份"信息部件"（§3.7），由若干视图组件拼成——记住：视图不是"描述"本身，架构描述才是描述。架构视角（§3.8）则是一组惯例，用来创建、解释、使用视图，把一

项或多项关注点（§3.10：对利益相关方而言有相关性或重要性的事项）框住；这组惯例写下来，就是视角规格（注 3）。两者关系是：一个视角管一个或多个视图，一个关注点也能由不止一个视角框住（§5.2.7）。至于 TOGAF 那四个"架构"——业务、数据、应用、技术，它们是四个关注域，是四个领域各自的设计方案，**既不是四个架构，也不是一个架构的四块拼图**；一个所关注实体只有一个架构，只是可以用一份或多份不同的架构描述来表达（§5.2.2）。

解析： TOGAF 自己把它们叫 architecture domains，"域"是在同一个对象上切出来的领域，跨域一致才是要答的题。另外，用来表达某个视角、某个方面或某个抽象层级的描述，标准里叫架构种类（42020:2019 §E.4.1.1），不叫视图——"安全架构、逻辑架构、物理架构"是种类，不是视图。多视图不等于多架构，这正是 42010:2022 的意思：架构一个，描述可以多个。为什么必须分视角？因为关注点天然打架——同一栋楼，住户要隔音、物业要管线，一张图满足不了所有人，所以分开画，拼起来才完整。

示例： 〔案例·架构·视图〕AD-01 里"故障件可追溯"这条基本属性，四个域各画一张：业务视图看拆换按件号登记，数据视图看主数据带件号与履历，应用视图看排故工单串起履历，技术视图看数据落库与审计。四张图合起来才是同一个架构，不是四个架构。

4. 开发架构过程，其输入的产品数据有哪些，其输出的产品数据有哪些

答案： 输入分两组六类（本课的归类，每类都能对上标准）。判据侧三样：目的与任务（来自业务或任务分析，15288:2023 §6.4.1，用《系统目的或任务理解确认单》锚住，排第一位）、规定需求（§3.36 / §6.4.3）、关注点（42010:2022 §3.10）。空间侧三样：约束、环境（§3.13）、资产（15288:2023 §3.37 资源 / §3.6 人工制品）。那张确认单是入场券——它是目的判据的锚，算输入、不算产出。输出九样（本课的清单，每样都能落到条款）：关注点台账（42010 §6.4）、架构目标与关键成功准则（42020:2019 §8.4.4）、视角规格集 / 项目 ADF（42010 §3.5 / §6.6）、基本概念和基本属性清单（42020 §8.4.8）、架构描述 AD（42010 §3.3 / §6.1）、决定清单 ADR（42010 §6.10）、管控原则（§3.2 定义句的后半）、系统接口契约（15288:2023 §3.19 / §6.4.4）、系统级验证方案（15288 §6.4.9）。

解析： 输入不是"把需求原文全搬过来"，而是按用途分两组：判据侧告诉架构师"什么才算基本"，目的与任务排第一，靠确认单锚住；空间侧圈出"在多大地盘里选"。产出则是把"基本"的裁决落成产品数据：清单是纲（登记什么配得上基本）、AD 按视图表达、ADR 留痕（记下"当时为什么这么选"）、管控原则管住实现和演化，再加上关注点台账、架构目标、视角规格集、接口契约、系统级验证方案——九样齐了，架构才既被裁决过、又被表达清楚、还能评审和追溯。提醒一句：42020:2019 §8.5 列的是架构概念化的工作产品；ADR 是业界的习惯叫法，标准里对应的条款是 §6.10"决策与理由的记录"。

示例： 〔案例·架构·输入输出〕入口先亮入场券：PU-01（目的与任务）、RQ-03（规定需求），加上客户与适航的关注点；再圈地盘——适航约束、现网环境、现有平台与数据资产。出去的是一整套：关注点台账 CN-01、架构目标 GO-01、视角规格集 VS-01、清单 CP-01、架构描述 AD-01、决策记录 ADR-01、管控原则 GP-01、接口契约 IC-01、系统级验证方案 VP-01。

5. 系统架构表示为基本概念和基本属性清单，对系统开发的作用是什么

答案： 这份清单是"目的判据"裁出来的结果（判据就是"对目的必要且充分"，IEV 831-01-01 Note 2 的二手说法），每一条都写着：定义是什么、为什么它对目的必要、能追到哪条目的或需求。它的作用有四个：①登记"什么配得上基本"，往后所有设计决策都有锚；②当架构评审的第一张核对单，从"这条配得上基本吗"问起；③把清单做成基线，增一条删一条都走变更（基线的定义见 15288:2023 §3.8）；④本质锁住以后，详细设计和实现可以放手展开——守住了不变，才换来了自由。

解析： 准入门槛是"必要性论证"：每一条都得答得出"对目的必要且充分吗"（去掉不是它、缺了垮掉、换了变质），并且挂上到目的、需求的追溯链。答得出的才配进清单；答不出的说明它到不了目的这个源头，是输入链上的"无源之水"，该剔。标准侧有两处支撑：架构评估要判"架构有没有应对利益相关方的关注点""架构实体有没有满足预期目的"（42030:2020 §3.4），清单正是这两问的底稿；架构决策与理由要记录（42010:2022 §6.10），清单的增删同理要留痕。问一答之间，"基本"就从"我觉得重要"变成了逐条可查的枚举。

示例： 〔案例·架构·清单〕CP-01 里"数据一次录入全程复用"是基本概念，附了必要性论证。后来有人以"赶节点"为由提出各系统各自录一遍——对着清单一看，这条一砍，追溯链就断、适航证据也拼不起来，论证答不过，变更在动工前就被挡下。

九、治理与管理辨析

1. 基于ISO标准，什么是治理，定义中有哪些主要元素，怎样理解它们

答案： 治理，就是一个组织里"谁定方向、谁盯执行、谁担后果"的那套系统。它的本体定义取 ISO 37000:2021 §3.1.1〔库外〕，三成分的说法取 ISO/IEC 38500:2024 §3.3〔库外〕：治理是以人为基础的系统，由指导、监督、担责构成。①指导，是定方向和规矩，把期望的目的和结果说清楚；②监督，是查执行——按既定的原则和目标核对，看绩效高低、有没有跑偏，发现偏差就处置；③担责，是对结果和行为向利益相关方交代、可被追责。这三件事里有一条铁律：**责任可授，担责不可授**（37000:2021 §4.2〔库外〕）。

解析： 把治理当成动词看更清楚：govern 就是定方向和规矩、查执行、担后果这三个动作。方向上也记一句：监督是往下看（看管理层），担责是往上担（对股东）。英文词要分清：定义句第三个成分是名词 accountability，监督对应的是 overseeing，而 monitor 是 §6.2.4 里的任务名——同一份标准里两个词都用，译的时候不能都叫"监督"。还有一层别混：37000 讲组织治理本身，38500 讲 IT 治理（是它的域应用），42010、42020 只提供治理所作用的那些工件和过程。

示例： 〔案例·治理·三成分〕董事会定方向（聚焦机载航电、三年不跨界）、按季度核查经营层、对股东交代结果——指导、监督、担责三件事各就各位。它不下场管设计评审，但"该不该做消费电子"这种问题，只有它能回答。

2. 为什么业务存在委托时，就必须要有治理，治理的目的究竟是什么呢

答案： 委托，就是治理机构把具体责任下放给管理层（ISO/IEC 38500:2024 §7.2.5〔库外〕）；但担责下不去——治理机构对组织整体负最终责任（§5.6〔库外〕）。所以才有那条铁律：责任可授，担责不可授。下放出去的事一旦脱离指导、监督、担责，宗旨就保不住；而最终担责仍旧落在治理机构身上。也正因如此，只要发生委托，治理就必须跟上：治理机构对被下放的那部分，继续行使指导、监督、担责。治理的目的有三个（38500 §4.1 / 37000 §4.1〔库外〕）：有效达成目的（绩效）、负责任地守护资源（受托）、合乎伦理行事（伦理）。

解析： 要注意：委托不是治理的前提——没有委托，治理同样是组织常态（§3.3）；同一个人兼着治理和管理两个角色时，也得分清此刻在履行哪一个。委托只是让治理从"常态"变成"不可回避的必需"。标准里"担责不可授"有两处同源说法：授权可以，但对组织的作为、不作为和渎职的责任，仍旧归治理机构（37000 §6.5.2 / 38500 §6.1〔库外〕）。至于代理鸿沟这类公司治理理论，不在标准之内，本讲不采用。

示例： 〔案例·治理·委托〕日常经营下放给总经理李承志，担责留在董事会：研发投入超营收 5% 要回董事会批，重大型号转阶段要报备。授权可以，甩责不行——所以只要有委托，治理就得跟上，而不是签个字就撒手。

3. 治理与管理的关系是什么，为什么管理不能取代治理，二者如何分工

答案： 治理（动词）= 定方向和规矩、查执行、担后果，也就是 ISO/IEC 38500:2024〔库外〕三成分的行使（名词定义见本组第 1 题）。管理是"指挥和控制组织的协调活动"（ISO 9000:2026 §3.2.1）。两者不是谁大谁小，也不是并列，而是一条纵向链：上面治理定方向和框架，下面管理在框架里执行、达成运

行目标。管理为什么取代不了治理？因为管理层自己就在框架里：自己定规则、自己执行、自己检查，回答不了"这规则该不该这么定"。治理由治理机构独立行使，最终担责也在治理机构。

解析： 管理层的"依据"其实正是治理的产出：权来自治理的授权，要向治理报告，还要受治理监督——这条纵向链就是管理存在的根据。管理干的活都在治理给的框架里；一旦自己定框架，就成了自授权、自裁判——规则制定者和执行者必须分开，自审自判没有公信力。标准侧还有个旁证：管理体系是组织中一组相互关联或相互作用的要素，用来确立方针和目标，以及实现这些目标的过程（ISO 9000:2026 §3.4.2）——它管的是"怎么达成"，不是"该不该这么定"。

示例： 〔案例·治理·分工〕董事会定"研发投入结构"这个边界，总经理在边界内决定投哪几个型号。总经理不能自己改边界、自己评边界——规则制定者与执行者必须分开，否则自查自评没有公信力。

4. 治理工作是怎么启动的和执行的，管理工作又是怎么启动和执行的呢

答案： 治理是持续运转的，不靠某个事件来"启动"。它按 ISO/IEC 38500:2024〔库外〕§6 的模型循环（§6.1 原则，加上 §6.2.1–§6.2.4 四个活动：利益相关方参与、评估、指导、监督；"相关方参与"是 §6 条款名的照录，本件行文写"利益相关方参与"）：先识别并让利益相关方参与进来→评估方向和执行情况→指导定方向和规矩→监督核对绩效与合规，然后回到开始，如此往复。所谓"启动了"，其实只是评估发现了偏差、触发了一轮处置。管理也一样持续运转：在治理给的框架里按 PDCA（计划—执行—检查—处置）走，不停地达成运行目标，也不设什么"启动"。

解析： 四活动和三成分别混：四活动（利益相关方参与、评估、指导、监督）讲的是治理怎么转，出自 §6 模型（2024 版比上一版新增了"利益相关方参与"）；三成分（指导、监督、担责）讲的是治理由什么构成，出自 §3.3。执行看模型，定义看三成分。标准里并不存在"启动"这一步：治理是常态系统，评估发现偏差就调整指导。管理侧的持续改进落在过程方法上（ISO 9000:2026 §3.3.4：识别、监视并管理过程及其相互作用）。至于治理与管理之别，看的是职能层级和对象（治理定框架、管理在框架内执行），不是看谁先动。

示例： 〔案例·治理·执行〕治理不是"开个会启动"：每季度让客户、适航代表、供应商参与进来（利益相关方参与），评估经营与合规表现，据此调整投入方向，再核查整改落实情况，转一圈再评估。管理侧则按 PDCA 天天跑：计划、执行、检查、处置。

5. 企业架构为什么必须治理，其目的和组织是什么，架构师职责是什么

答案： 企业架构就是企业运营体系架构。它为什么必须治理？因为架构好不好，由运营体系的宗旨来评判，而宗旨握在治理机构手里，不在执行者手里。架构治理的定义在 ISO/IEC/IEEE 42020:2019 附录 G.1（资料性）里说得很直白：它是一致的管理、协调的标准与方针、恰当的指导、统一的过程和适当的决策权所必需的，针对给定的职责领域实施，为的是确保恰当的监督与担责；过程正本在第 6 章（§6.4.1–§6.4.6 六项任务、§6.5 工作产品）。架构和宗旨的一致不会自己保持：战略一调整，宗旨跟着变，原架构可能就不合格了；运营中也会慢慢偏，架构自己不会纠。所以要由治理机构持续指导、监督、担责，把架构按在宗旨上。组织上，是治理机构下的架构委员会（G.1 里叫"设计权威机构"）。架构师干什么？干架构工作（42010:2022 §3.1：构想、定义、表达、编制文档、沟通、证实其正确实施、维护和改进架构），做架构评估（42030:2020 §3.4：针对评估目标对架构作出判断）与符合性判断，发现偏差就上报治理机构；他不定义宗旨——那是治理机构的事。

解析： 四条理由：①架构的价值在服务宗旨，评判权就跟着宗旨落在治理机构；②治理管的是"守还是变"的边界：架构要稳、宗旨在变，谁裁决该守还是该变；③架构错了很难回头，落地后改动代价极大，所

以治理得前置；④宗旨会变，除了战略调整，日常一个个局部决策也会把它带偏。治理的目的就一句：让架构持续符合运营宗旨（过程见 42020:2019 第 6 章；42030:2020 §3.4 把"是否满足预期目的"列为架构评估的判断之一）。

示例： 〔案例·架构治理〕宗旨从"航电研制交付"漂向"消费电子代工"时，出问题的不是技术，是架构没跟上宗旨：委员会确认新宗旨，架构师把偏差报上去（架构工作本来就是构想、定义、表达、编制文档、沟通、证实实施、维护和改进），架构跟着宗旨校正。

十、学习与概念体系

1. 什么是概念，所指、能指和指称各是什么，它们之中哪个才是概念呢

答案： 概念是头脑里的一个"知识单元"，由一组特征以独特的方式组合而成（ISO 1087:2019 §3.2.7〔库外〕：unit of knowledge created by a unique combination of characteristics；本库译"由特性的独特组合而形成的知识单元"，不是官方中译——国标旧版作"由特征的独特组合"，而且对应的是已废止的1087-1:2000）。用大白话说，概念是对一类对象共同特征的抽象概括。能指是词或符号，就是概念的名字；所指是概念本身，即头脑里那团意思；指称是概念指向的客观事物。三者里面，概念是所指、是被指称的那团思想内容——它既不是词，也不是物。

解析： 三条线索别搅成一条：索绪尔说概念就是所指（signified），跟能指（signifier，词）相对；弗雷格说概念是涵义（sense），不等于指称（reference，对象）；至于"语义三角"（符号—思想—事物），那是奥格登与理查兹的说法，属普通语言学引述、不是标准条文。三者讲的都是"概念在符号系统里站在哪儿"，都没另立概念的定义。一句话记住：概念在头脑里（所指），不在嘴上（不是能指），也不在物上（不是指称）。把词当成概念，正是"同一个词、各说各话"的根源。

示例： 〔案例·概念·所指〕会上三个人都说"数据"：小林指模型文件，老黄指机队遥测流，周经理指 KPI 数字——同一个词、三个所指、三个概念。先把各自说的是哪个概念分清，再谈"数据质量"，否则吵一晚上也吵不到一处。

2. 概念体系中，什么是根概念、核心概念和一般概念，它们是什么关系

答案： 一个领域的概念可分三层（本课的分法）：一般概念，是这个领域里所有能指称、能界定的概念；核心概念，是枢纽——别的概念大多靠它来定义；根概念，是再没有别的概念能定义它、其余概念都由它派生的那个。三层是层层包含的：一般包含核心，核心包含根，范围逐层收窄、判据逐层收严、数量逐层变少。注意分层的依据是网络地位，不是重要程度——重要不等于源头。"被引用得最多"只是核心概念一个好用的代理判据（便于统计），不是它的定义。

解析： 核心和根的区别在于地位，不在于重要性：被引用得多指向核心，不再被任何概念定义才是根。一个概念再重要，只要还得靠别的概念定义，它就不是根。标准侧给的是概念之间的关系语汇（ISO 9000:2026 附录 A）：属种关系（A.2，上位与下位）、整体部分关系（A.3）、联想关系（A.4）。"根—核心—一般"这三层，其实就是沿属种关系一路往上追：追到再没有上位概念的地方，那就是根。

示例： 〔案例·概念·三层〕在华翔这个领域：库存记录、适航记录是一般概念；流程、组织、绩效、需求被反复引用，是核心概念；产品数据不靠别的概念定义、其余都由它派生，是根概念。需求很重要，但它是核心概念，不是根。

3. 掌握根概念和核心概念，对理解和应用该领域的专业知识有什么作用

答案： 抓住根概念，就抓住了这个领域的源头：大多数概念由它派生，学新概念时能一路追到出处和依据。抓住核心概念，就抓住了枢纽：大多概念靠它定义，认识了它，学新东西就省力。两样合起来就是理解这个领域的地基——所以学习的顺序应该是先抓根和核心，其余概念用到时再补。

解析： 为什么先抓这两层？因为被依赖得最多：核心概念是大多数概念的参照，不懂它，一堆概念没法定义；根概念是源头，不懂它，概念之间的来龙去脉说不清。其余概念可以由根和核心推出来，不必先背全。标准侧的机制解释也在这儿：概念之间靠属种关系构成层级，下位概念继承上位概念的全部特征，再

加上自己的区分特征（ISO 9000:2026 附录 A.2）——所以上位一变，下位的定义跟着变；先立上位，学习和沟通的成本最低。

示例： 〔案例·概念·作用〕新来的小赵先学根（产品数据）和核心（流程、需求），再看"适航记录""库存管理"这些一般概念，会发现都能挂回上层——这叫学得下去。反过来，从一条条日志表学起，学半年也拼不出全局。

4. 你是怎么学习理解一个概念的，有何判据可以判定你掌握了这个概念

答案： 掌握的标准是会用：背得出定义不算，能拿它做判断、解决问题才算。学一个概念可以分五步：先弄清它在领域里站在哪个位置；再跟自己已有的知识接上；然后讲给别人听，讲得清才算真懂；再对着事实核对，错了就改；最后放进真实任务里反复用。判定的法子是四问：能答出它是什么、不是什么、从哪来、往哪去，才算拥有；只答得出"是什么"，那只是见过。

解析： "掌握 = 会用"这条，依据是布鲁姆认知目标分类学：记住、理解之后，走到"应用"才算真掌握；维特根斯坦那句"意义即用法"属哲学引述、不是标准条文——两者指向同一个判据：能判断、能迁移，而不是能背。标准侧还能给"定义"本身立个判据：定义是用描述性短语对一个概念的表述（ISO 1087:2019 §3.3.1〔库外〕）——所以"讲得清"不是口才问题，是能不能给出合于概念的表述；"不是什么"对应定义里的区分特征，"从哪来"对应它在属种关系里的上位概念。五步和四问法是本课的方法论。

示例： 〔案例·概念·掌握〕背得出"质量 = 固有特性满足要求的程度"不算掌握；能拿它评一份验收标准、能判断一次"告警延迟"算不算质量问题，才算掌握。老张带徒弟就用四问：它是什么、不是什么、从哪来、往哪去。

5. 给出运营体系这一领域的根概念与核心概念，及它们为什么是的理由

答案： 在运营体系（企业运行）这个领域，根概念是产品数据（这是本课在给定领域边界下的结论，换一个领域边界，根会变）。理由：企业运行里跨部门、跨阶段流转的恰恰是产品数据；流程、组织、绩效、质量都由它派生，而它不依赖其中任何一个；数据一丢，运营就停。核心概念是流程、组织、绩效、需求、质量、治理这些：它们被反复引用，但都能由产品数据派生——流程是数据流动的方式，组织是数据的分工归属，绩效是衡量目标达成程度的数据。它们都不是根。

解析： 为什么根落在产品数据而不是产品？因为企业运行层面，产品是以数据形态存在的（ISO 10303-1:2021〔库外〕讲的就是产品数据是产品的信息形态）。标准术语链也是这个走向：数据是关于客体的事实（ISO 9000:2026 §3.8.1），信息是有意义的数据（§3.8.4），客体是"可感知或可想象到的任何事物"（§3.5.3）——所以"以数据为载体"不是修辞，是术语链的落点。流程也不够格当根：它只是数据流动的一种方式，可以重画，数据本身不变。

示例： 〔案例·概念·根〕华翔可以重画研发流程——流程变了，产品数据还在；可数据一旦丢了、对不上，研发当天就得停。所以流程是核心概念，产品数据才是根：动流程是改方式，动数据是动命根。

---

*生成日期：2026-08-30；**v2.3 修订：2026-10-20**——① 引用面收口为**标准条款**，不引任何内部研究资料；② 语体改为**先讲人话、再给标准原文**：结论用大白话说，标准定义与条款号一律进括注，本课自编的讲法（三态、八方案、九样、六项能力、四化递进、根 / 核心 / 一般概念）句中直接说明"这是本课的讲法"；③ **50 个示例改为一条贯穿案例**（华翔研发运营体系）：统一企业、体系、五方、人物与工件编码，示例开头标〔案例·位置〕，文前设"案例底稿"（企业 / 四大组成 / 三态生命线 / 五方 / 工件编码表 / 示例路线图），使示例既解释当题术语，又让人对运营体系的**组成、开发过程、运行**形成整体框架与感性认*

识。对应《01-运营体系正向设计基础概念》（10 组 × 5 问 = 50 题），每题含【答案】【解析】【示例】。